

Contents

Plan: Multi-Fault Conformal Mesh via Cross-Fault Intersection Resolution	1
Status	1
Data flow at a glance	2
Inputs (what we read)	2
Processing (what we do)	3
Outputs (what we write)	3
What the pipeline does NOT change	4
Why this plan exists	4
Approach (one-paragraph summary)	4
Constraints	4
File layout (new files only; existing files modified per the review)	5
Phase 1 — Cross-fault intersection geometry	5
Goal	5
Files to create	5
Public API	5
Algorithm	6
Acceptance criteria	11
Edge cases to handle	11
Phase 2 — Constrained re-triangulation	11
Goal	11
Files to create	11
Public API	11
Algorithm	12
Robustness rules	15
Validation gates (per fault and per pair)	15
Acceptance criteria	15
Edge cases to handle	16
Phase 3 — Tube-uniform size field	16
Goal	16
Files to modify	16
Acceptance criteria	17
Edge cases to handle	17
Phase 4 — End-to-end 2-fault target	17
Goal	17
Files to create	17
Files to modify	17
Acceptance criteria	18
Phase 5 — Cascade to 3+ faults (out of scope; verification only)	18
Goal	18
Files to modify (verification only)	18
Acceptance criteria	18
Testing strategy	18
Risk assessment	19
Out of scope (explicitly)	20

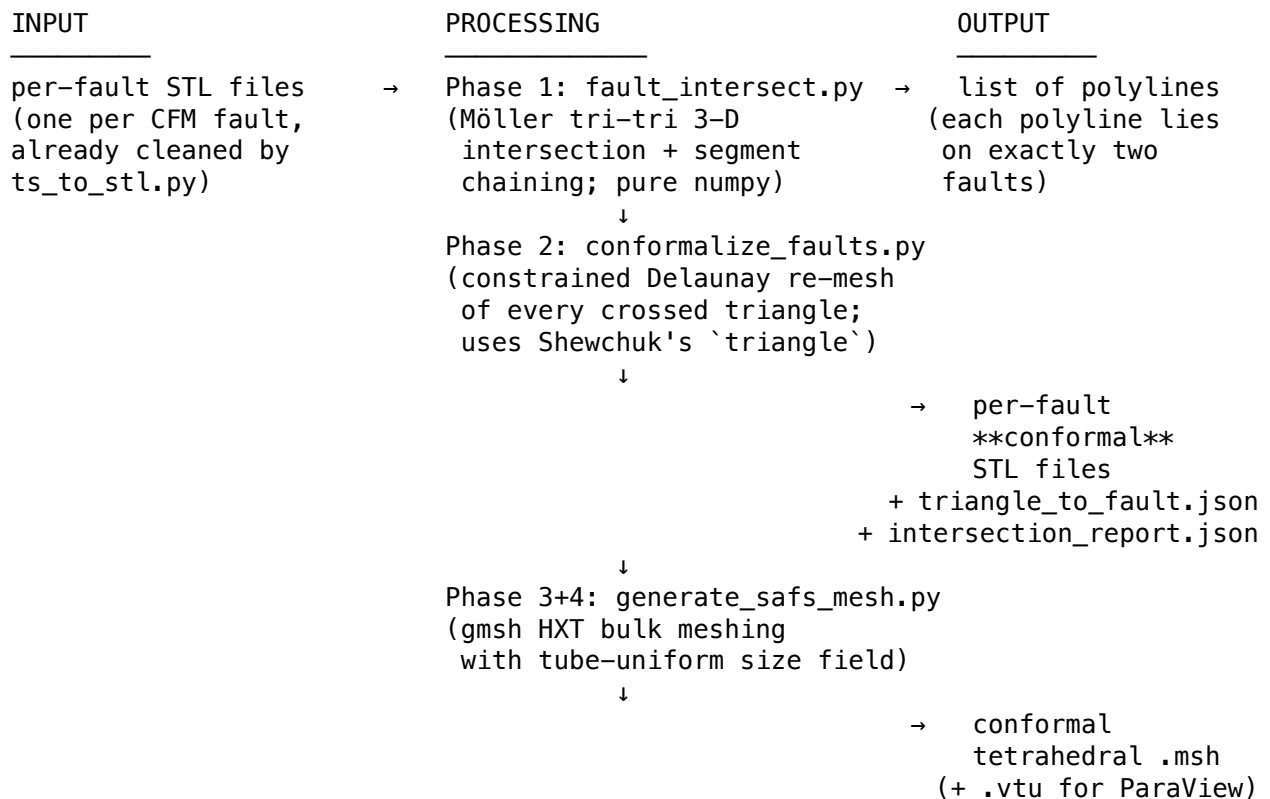
Plan: Multi-Fault Conformal Mesh via Cross-Fault Intersection Resolution

Status

- Created: 2026-04-29.

- **Companion review:** mesh/REVIEW.md (findings R-001 .. R-006).
- **Companion plans:** PLAN.md (overall pipeline), PLAN_domain.md (size field & tags), PLAN_origin.md (coordinate frame), PLAN_smoke_millcreek.md (single-fault smoke test).
- **Driving requirement (user, 2026-04-29):** > “the current workflow fails to generate conforming mesh; ... each fault > has its own point cloud, which has no prior knowledge about whether > other fault trace will intersect with it; with all point clouds > input, we should find a way to identify the intersection locations > and properly handles it such that the mesh is conforming and > simulation can adopt; ... keep near the fault region mesh size to be > uniform ... to test our approach take just two fault traces (that are > intersecting at some places).”
- **First milestone:** run_two_crossing_2000m.sh passes 10/10 validator checks on Mill Creek × SBMT San Andreas at 2000 m, with a tube-uniform near-fault region.

Data flow at a glance



Inputs (what we read)

File	Format	Contents	Source
<stl_dir>/<short>.stl	ASCII STL	Per-fault triangulated surface in the SAFS local frame (metres)	ts_to_stl.py (existing) — cleaned + clamped to free surface
transform.json	JSON	UTM origin, free-surface clearance	ts_to_stl.py

File	Format	Contents	Source
bbox.json	JSON	Per-fault local-frame bounding boxes	ts_to_stl.py

Each fault's STL is **independent** — there's no shared topology, no shared vertices, no notion of "where fault A meets fault B". Detecting those crossings is the first job of this pipeline.

Processing (what we do)

Stage	Tool / library	What it does
1. Detect crossings	numpy (Möller's algorithm, written in fault_intersect.py)	Loops over triangle pairs from different faults, returns 3-D segments where they cross
1b. Chain segments	numpy graph traversal	Stitches segments end-to-end into polylines
2a. Project to 2-D	numpy linear algebra	Each crossed parent triangle is projected to its own plane
2b. Re-mesh	triangle (Shewchuk) — wrapped via pip install triangle	Constrained Delaunay triangulation, flag 'p' only
2c. Propagate	numpy	Splits each pierced edge's neighbour triangle so no T-junctions remain
3. Bulk mesh	gmsh HXT (Algorithm3D=10) — invoked by generate_safs_mesh.py	Generates conforming tetrahedra with tube-uniform size field
4. Validate	meshio + the validator's 11 checks	Confirms every fault triangle has 2 adjacent tets (the SEAS DG requirement)

Outputs (what we write)

File	Format	Contents	Consumer
<out_stl_dir>/<short>ASSTL	ASCII STL	Conformal per-fault STL (every cross-fault polyline appears as a sequence of triangle edges)	generate_safs_mesh.py Merge step
triangle_to_fault.json	JSON	Per-triangle fault attribution (range-based; survives later splits exactly)	write_fault_provenance.py (replaces its KDTree path)
intersection_report.json	JSON	Per polyline: faults, # pierce points, # children added on each side; aggregate counts	Human review / debugging

File	Format	Contents	Consumer
safs_<run>.msh	Gmsh .msh v2.2 ASCII	Tetrahedral bulk mesh with internal fault interfaces, tube-uniform near-fault sizing	MFEM SEAS solver
validation_report.txt	text	11/11 (or 10/10) check pass/fail with metrics	CI / human gate
safs_<run>.vtu	VTK	Same mesh, for ParaView visualisation of fault tags & sizing	Human review

What the pipeline does NOT change

- The **CFM .ts source files** are read once at the very top (by `ts_to_stl.py`) and never touched again. All transformations are produced as new files.
- The **single-fault smoke test** still runs unchanged — when there's only one fault in `--include-fault`, the new conformalize step is a no-op pass-through.
- The **bulk solver** (BP5/BP1/BP2 source) is not modified — only the upstream mesh generation changes.

Why this plan exists

PLAN.md Phase 4 specified intersection resolution via OCC BooleanFragments. That path is dead (gmsh 4.13 cannot promote discrete STL surfaces to OCC NURBS) and the HXT discrete-embedding fallback rejects multi-fault input with PLC errors. The current code silently emits a non-conforming mesh — `output/all8_2000m/output/validation_report.txt` shows every one of 4917 fault triangles is detached from the tet mesh (`n_internal_with_2_tets: 0`). This plan replaces Phase 4 with a custom geometric pipeline.

Approach (one-paragraph summary)

Each CFM fault arrives as an independent triangulated point cloud. We add a pre-meshing step that (1) finds every triangle-pair from different faults that intersects in 3-D, (2) chains the pairwise intersection segments into polylines, (3) re-triangulates each crossed triangle with constrained Delaunay so the polyline becomes an explicit edge sequence on both faults, and (4) hands the now-conforming PLC to gmsh's HXT mesher. The bulk size field is generalized to a tube-uniform form so the near-fault zone is uniform at `res_f` rather than ramping immediately. We prove the pipeline on the smallest non-trivial input — two CFM faults that genuinely cross — before scaling.

Constraints

- **No edits to BP5/BP1/BP2 sources.** Per project memory [C2] (`feedback_tpv102_bp5_no_shared_edit.md`) and per `feedback_dynamic_folder_editable_for_tpv104.md`, the SAFS work is isolated to `miniapps/seas/safs/`.
- **pythonenv for everything.** Per `seas-mfem/CLAUDE.md`. Allowed packages: `numpy`, `scipy`, `meshio`, `gmsh`, `jsonschema`, and the triangle bindings around Shewchuk's library (declare in `pythonenv` if not already present; `pure-pip`).
- **Reproducibility.** All numerical tolerances live in CLI flags or named module constants; no hidden magic numbers.

- **Defensive failure.** Multi-fault input that has not been conformalized must be rejected by the driver before gmsh runs (R-002).
- **Provenance preservation.** Per-triangle fault attribution must survive every split (R-006).
- **Coordinate frame unchanged.** All work is in the SAFS local frame defined by `safs_origin.py`. No re-translation.

File layout (new files only; existing files modified per the review)

```

miniapps/seas/safs/
├── PLAN_multifault_intersections.md    ← this document
├── mesh/
│   ├── fault_intersect.py             ← Phase 1: tri-tri 3D intersection
│   ├── conformalize_faults.py         ← Phase 2: constrained re-triangulation
│   ├── tests/
│   │   ├── test_fault_intersect.py    ← synthetic + real fixture tests
│   │   └── test_conformalize_faults.py
│   ├── run_two_crossing_2000m.sh      ← Phase 4: 2-fault end-to-end
│   └── output/two_crossing_2000m/     ← (generated) per-run artefacts

```

Phase 1 — Cross-fault intersection geometry

Goal

After this phase, given a directory of cleaned per-fault STLs and a list of included faults, we can emit every cross-fault triangle pair that intersects in 3-D, plus the segment of intersection.

Files to create

- `mesh/fault_intersect.py` — pure-Python, numpy-only.
- `mesh/tests/test_fault_intersect.py`.

Public API

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Segment:
    p0: tuple[float, float, float] # local-frame metres
    p1: tuple[float, float, float]
    tri_a: int                     # triangle index in fault_a
    tri_b: int                     # triangle index in fault_b
    fault_a: str                   # short_name
    fault_b: str
    nondegenerate: bool           # |p1-p0| >= eps_min_seg_len_m
```

```
@dataclass
```

```
class Polyline:
    points: list[tuple[float, float, float]]
    closed: bool
    fault_a: str                   # both faults that this curve lies on
    fault_b: str
```

```

pierces_a: list[int]                # tri indices in fault_a that this curve crosses
pierces_b: list[int]

def tri_tri_intersect_3d(
    V_A: np.ndarray, T_A: np.ndarray,
    V_B: np.ndarray, T_B: np.ndarray,
    fault_a: str, fault_b: str,
    eps_orient: float = 1e-9,
    eps_min_seg_len_m: float = 5e-2,    # 5 cm – must exceed 2 * snap_m
    clearance_m: float = 0.0,          # free-surface clamp (Phase 1 step 9)
) -> list[Segment]: ...

def chain_segments(
    segments: list[Segment],
    snap_m: float = 1e-2,              # 1 cm – must satisfy snap_m < eps_min_seg_len_m / 2
) -> list[Polyline]:
    """Pre-condition: every input segment satisfies
     $|p_1 - p_0| > 2 * \text{snap}_m$ . Otherwise it would collapse to a
    self-loop after endpoint snapping. The function asserts this
    on entry."""
    ...

def scan_cross_fault_crossings(
    stl_dir: Path,
    included: list[str],
    clearance_m: float = 0.0,          # forwarded to tri_tri_intersect_3d step 9
) -> dict[tuple[str, str], list[Polyline]]:
    """Used by R-002's gate in generate_safs_mesh.py – returns {} when
    every pair is non-crossing (the disjoint-subsets case). When
    called from the conformalize driver, pass the same clearance
    that ts_to_stl.py used so polyline endpoints are clamped at
     $z = -\text{clearance}$  rather than truncated."""

```

Algorithm

Triangle–triangle intersection (Möller's method).

One-paragraph idea Two non-coplanar triangles, if they intersect, intersect along a **straight line segment** that lies in both triangles. The line extending that segment is $L = \text{plane}_A \cap \text{plane}_B$. Each triangle contributes its own interval on L (the piece of L that is inside the triangle). The intersection segment is the **overlap of those two intervals**. So the whole problem reduces to: find L , find each triangle's interval on L , take the overlap.

Picture (1–D, along the intersection line L) Stop thinking about the two triangles in 3–D. The whole algorithm happens on a single line — the line L where the two planes meet.

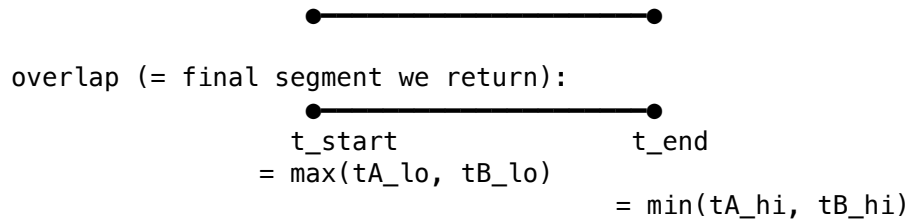
direction D along L $\longrightarrow$

triangle A's interval on L :



triangle B's interval on L :





Each triangle, projected onto L, occupies an interval. The intersection segment is where the two intervals overlap. If they don't overlap (e.g., A's interval ends before B's begins), there is no intersection.

Step-by-step

INPUT: triangle A = (A0, A1, A2)
triangle B = (B0, B1, B2)

OUTPUT: segment (p_start, p_end) on the line L = plane_A intersected with plane_B, or None if they don't intersect.

(1) **Quick reject — is A entirely on one side of plane B?** — $n_B = (B1 - B0) \times (B2 - B0)$ (normal of plane B; not normalised) — $d_B = -n_B \cdot B0$ (plane offset, so plane B = {x : $n_B \cdot x + d_B = 0$ }) — For each i, signed distance $d_{Ai} = n_B \cdot Ai + d_B$. — If d_{A0}, d_{A1}, d_{A2} are all > 0 or all < 0 , A doesn't reach plane B => no intersection, return None.

(2) **Quick reject — is B entirely on one side of plane A?** — Symmetric: compute d_{Bj} for $j = 0, 1, 2$. Same all-same-sign test.

(3) **Build the intersection line L.** — Direction: $D = n_A \times n_B$ (unit-normalised). D is by construction parallel to both planes, so it lies along L. — We don't need a base point on L; the next step parametrises positions along D directly.

(4) Where does triangle A meet line L?

Triangle A's three vertices have signed distances d_{A0}, d_{A1}, d_{A2} to plane B. By step (1) they don't all have the same sign, so exactly **two of A's three edges cross plane B** (the third edge has both endpoints on the same side). Each crossing edge gives one point of A intersected with plane B, and that point lies on line L as well (since plane A intersected with plane B = L, and the crossing points lie in plane A).

For each of those two edges (A_i, A_j) whose signed distances differ in sign, linearly interpolate:

$$\begin{array}{ll}
 s = d_{Ai} / (d_{Ai} - d_{Aj}) & \text{in } [0, 1] \\
 p_{\text{hit}} = A_i + s \cdot (A_j - A_i) & \text{in } \mathbb{R}^3, \text{ lies on L} \\
 t = D \cdot p_{\text{hit}} & \text{scalar parameter on L}
 \end{array}$$

Call the two t-values $tA_{\text{lo}} = \min(t1, t2), tA_{\text{hi}} = \max(t1, t2)$. Now A's footprint on L is the interval $[tA_{\text{lo}}, tA_{\text{hi}}]$.

(5) **Same for triangle B**, giving $[tB_{\text{lo}}, tB_{\text{hi}}]$.

(6) Overlap.

```

t_start = max(tA_lo, tB_lo)
t_end   = min(tA_hi, tB_hi)
if t_start > t_end:      # disjoint intervals → no intersection
    return None

```

Recover 3-D endpoints — explicit formula. Each triangle has two p_{hit} points (its interpolated edge crossings at parameters $tA_{\text{lo}}, tA_{\text{hi}}$ for A, similarly for B). Both triangles' chords are subsets of the same line L, so they pass through the same 3-D point at any given parameter t. Pick triangle A's chord (or B's — same answer) and use the standard 1-parameter linear interpolation:

For an endpoint at parameter t in $[tA_lo, tA_hi]$:

$$\alpha = (t - tA_lo) / (tA_hi - tA_lo) \quad \# \text{ in } [0, 1]$$

$$p(t) = (1 - \alpha) \cdot p_hit_A_lo + \alpha \cdot p_hit_A_hi \quad \# \text{ 3-D position on } L$$

Apply this at $t = t_start$ and $t = t_end$ to get p_start and p_end .

Fast path: when $t_start == tA_lo$ (or tA_hi, tB_lo, tB_hi), α reduces to 0 or 1 and $p(t)$ is exactly that boundary's p_hit ; returning the matching p_hit directly is equivalent and skips the multiply-add. **Do NOT** confuse the fast path with “always return the p_hit of whichever t -value bounded the overlap” — that shortcut is correct only at the four boundary t -values; the formula above is what handles the general case.

(7) Length filter.

```
if |p_end - p_start| < eps_min_seg_len_m:
    mark nondegenerate=False; skip
```

(8) Free-surface clamp (parameter `clearance_m`, read from `transform.json`; default 0).

```
for endpoint p in {p_start, p_end}:
    if p.z > -clearance_m:
        # walk down the segment to z = -clearance_m
        alpha = (-clearance_m - p.z) / (p_other.z - p.z)
        p = p + alpha * (p_other - p)
if both endpoints now have z = -clearance_m
    AND the original segment's z-range was entirely > -clearance_m:
    drop the segment # it lies wholly above the bulk's free surface
```

Worked numeric example (sanity check for the implementer) Strictly non-degenerate inputs — every signed distance is non-zero, so no edge-case branches are exercised:

$A = (-5, -1, -5), (5, -1, -5), (0, 3, 5)$
 $B = (0, -2, -2), (0, 2, -2), (0, 0, 2)$

Step (1): $n_B = (B1-B0) \times (B2-B0) = (0, 4, 0) \times (0, 2, 4) = (16, 0, 0)$
 $d_B = -n_B \cdot B0 = -16 \cdot 0 = 0$
 $d_{A0} = 16 \cdot (-5) = -80 \rightarrow \text{negative}$
 $d_{A1} = 16 \cdot (5) = +80 \rightarrow \text{positive}$
 $d_{A2} = 16 \cdot (0) = 0 \quad \triangle \text{ would be zero} - N0: A2.x = 0, \text{ but we picked } A2 = (0, 3, 5) \text{ on plane } B \text{ again. Fix the example below.}$

The point on plane B ($x = 0$) is the source of degeneracy. Move A 's third vertex off $x = 0$:

$A = (-5, -1, -5), (5, -1, -5), (1, 3, 5)$
 $B = (0, -2, -2), (0, 2, -2), (0, 0, 2)$

Step (1): $n_B = (B1-B0) \times (B2-B0) = (0, 4, 0) \times (0, 2, 4) = (16, 0, 0)$
 $d_B = 0$ (plane B is $x = 0$)
 $d_{A0} = 16 \cdot (-5) = -80 \rightarrow \text{negative}$
 $d_{A1} = 16 \cdot (5) = +80 \rightarrow \text{positive}$
 $d_{A2} = 16 \cdot (1) = +16 \rightarrow \text{positive} \quad \checkmark \text{ all signed distances non-zero}$
 Sign pattern $(-, +, +)$: not all same $\rightarrow$ keep going.

Step (2): Compute $n_A = (A1-A0) \times (A2-A0) = (10, 0, 0) \times (6, 4, 10)$
 $= (0 \cdot 10 - 0 \cdot 4, 0 \cdot 6 - 10 \cdot 10, 10 \cdot 4 - 0 \cdot 6)$
 $= (0, -100, 40)$
 $d_A = -n_A \cdot A0 = -(0 \cdot (-5) + (-100) \cdot (-1) + 40 \cdot (-5))$

$$\begin{aligned}
&= -(0 + 100 - 200) = 100 \\
d_{B0} &= n_A \cdot B0 + d_A = (0, -100, 40) \cdot (0, -2, -2) + 100 \\
&= 0 + 200 - 80 + 100 = +220 \rightarrow \text{positive} \\
d_{B1} &= (0, -100, 40) \cdot (0, 2, -2) + 100 \\
&= 0 - 200 - 80 + 100 = -180 \rightarrow \text{negative} \\
d_{B2} &= (0, -100, 40) \cdot (0, 0, 2) + 100 \\
&= 0 + 0 + 80 + 100 = +180 \rightarrow \text{positive} \\
&\text{Sign pattern (+, -, +): not all same} \rightarrow \text{keep going.}
\end{aligned}$$

Step (3): $D = n_A \times n_B = (0, -100, 40) \times (16, 0, 0)$

$$\begin{aligned}
&= ((-100) \cdot 0 - 40 \cdot 0, \quad 40 \cdot 16 - 0 \cdot 0, \quad 0 \cdot 0 - (-100) \cdot 16) \\
&= (0, 640, 1600) \\
|D| &= \sqrt{0 + 640^2 + 1600^2} = \sqrt{409600 + 2560000} \\
&= \sqrt{2969600} \approx 1723.25 \\
\text{unit } D &\approx (0, 0.3714, 0.9285)
\end{aligned}$$

Step (4): A's two straddling edges (where d_A changes sign):

edge (A0, A1): $d_{A0} = -80, d_{A1} = +80$

$$\begin{aligned}
s &= -80 / (-80 - 80) = 0.5 \\
p_{\text{hit}} &= A0 + 0.5 \cdot (A1 - A0) = (0, -1, -5) \\
t1 &= D \cdot p_{\text{hit}} / |D| \approx (0 \cdot 0 + 0.3714 \cdot (-1) + 0.9285 \cdot (-5)) \\
&\approx -0.3714 - 4.6425 = -5.0139 \\
\text{edge (A0, A2): } d_{A0} &= -80, d_{A2} = +16 \\
s &= -80 / (-80 - 16) = 0.8333 \\
p_{\text{hit}} &= A0 + 0.8333 \cdot (A2 - A0) = (0, 2.333, 3.333) \\
t2 &\approx (0 + 0.3714 \cdot 2.333 + 0.9285 \cdot 3.333) \approx 0.8666 + 3.0950 \approx 3.9616 \\
tA_{\text{lo}} &\approx -5.0139, tA_{\text{hi}} \approx 3.9616
\end{aligned}$$

Step (5): B's two straddling edges (where d_B changes sign):

edge (B0, B1): $d_{B0} = +220, d_{B1} = -180$

$$\begin{aligned}
s &= 220 / (220 - (-180)) = 0.55 \\
p_{\text{hit}} &= B0 + 0.55 \cdot (B1 - B0) = (0, 0.2, -2) \\
t1 &\approx (0 + 0.3714 \cdot 0.2 + 0.9285 \cdot (-2)) \approx 0.0743 - 1.8570 \approx -1.7827 \\
\text{edge (B1, B2): } d_{B1} &= -180, d_{B2} = +180 \\
s &= -180 / (-180 - 180) = 0.5 \\
p_{\text{hit}} &= B1 + 0.5 \cdot (B2 - B1) = (0, 1, 0) \\
t2 &\approx (0 + 0.3714 \cdot 1 + 0.9285 \cdot 0) \approx 0.3714 \\
tB_{\text{lo}} &\approx -1.7827, tB_{\text{hi}} \approx 0.3714
\end{aligned}$$

Step (6): overlap interval = $[\max(-5.0139, -1.7827), \min(3.9616, 0.3714)]$

$$= [-1.7827, 0.3714]$$

Recover 3-D endpoints – both endpoints come from B's interpolation in this case (B's interval is fully inside A's):

$$\begin{aligned}
p_{\text{start}} &= (0, 0.2, -2) && (= B's p_{\text{hit}} \text{ at } tB_{\text{lo}}) \\
p_{\text{end}} &= (0, 1.0, 0) && (= B's p_{\text{hit}} \text{ at } tB_{\text{hi}})
\end{aligned}$$

Equivalently via the formula on A's chord at $t = -1.7827$:

$$\begin{aligned}
\alpha &= (-1.7827 - (-5.0139)) / (3.9616 - (-5.0139)) \\
&= 3.2312 / 8.9755 \approx 0.36 \\
p &= 0.64 \cdot (0, -1, -5) + 0.36 \cdot (0, 2.333, 3.333) \approx (0, 0.2, -2) \checkmark
\end{aligned}$$

Step (7): length = $\sqrt{0 + 0.64 + 4} \approx 2.155 \text{ m} \gg 0.05 \rightarrow \text{keep.}$

Step (8): clearance_m = 0 $\rightarrow$ no clamp.

RESULT: segment from (0, 0.2, -2) to (0, 1, 0). Approx 2.16 m long,
lies entirely on $x = 0$ (plane B) ✓.

If your implementation returns a segment whose endpoints have non-zero x , or whose length is closer to 9 m (the full A-piercing extent), you have implemented the discarded “clip A’s plane-piercing segment against itself” form — go back to step (6).

Acceleration: build an AABB tree over T_B (or use `scipy.spatial.cKDTree` on triangle centroids with `bbox-radius` pruning). For Mill Creek $\times$ SBMT-SAF (794 $\times$ 1346 = 1.07M pairs), naive $O(n^2)$ is fine (~1 s in Python with numpy broadcasting); for 8-fault all-pair the quadratic loop is acceptable too at this resolution. Defer the AABB tree until the naive run time exceeds 30 s.

Segment chaining: build a graph where each segment endpoint is a node (snapped to `snap_m` for matching), each segment is an edge. Connected components $\rightarrow$ polylines. Order endpoints along each polyline by following the edge chain. Detect closed loops (start == end).

Tolerance ordering invariant: the segment-length filter must use `eps_min_seg_len_m > 2 * snap_m` so that any segment surviving the length filter has endpoints distinguishable under the chain’s endpoint snapping. Defaults (5e-2, 1e-2) satisfy this with a 2.5 $\times$ margin. `chain_segments` asserts the invariant on its inputs. With the inverted ordering (`eps_min_seg_len_m < snap_m`), a segment shorter than `snap_m` would survive the length filter and chain to a zero-length self-loop, which the triangle library rejects.

Numerical robustness: – Default predicates use `np.float64`. When the orientation magnitude for any vertex of A relative to plane B is below the dimensionally correct threshold: – if you compute the **signed distance** $d_{Ai} = n_B \cdot A_i + d_B$ (units of length, **after** dividing by $|n_B|$), the threshold is `eps_orient * mean_edge` (also length). – if you compute the **unnormalised orient3d determinant** $(B_1-B_0) \times (B_2-B_0) \cdot (A_i-B_0)$ (units of length³, = 6 $\times$ tetrahedron volume), the threshold is `eps_orient * mean_edge**3`. – if you compute $n_B \cdot A_i + d_B$ without dividing by $|n_B|$ (units of length²), the threshold is `eps_orient * mean_edge**2`.

Pick one quantity and use the matching threshold. The original wording “`eps_orient * mean_edge_squared`” implicitly assumed the un-normalised- n_B form; an implementer who divides by $|n_B|$ must scale the threshold by `1 / mean_edge`, and an implementer who computes the full orient3d determinant must scale by `mean_edge`.

Mark the case as ambiguous and fall back to **exact arithmetic**. Two acceptable implementations (pick whichever is in `pythonenv`; if neither is, declare a hard dependency on `gmpy2` and use option (a)):

- (a) **gmpy2.mpq (rationals).** Convert the 12 input coordinates via `mpq(int(round(x * 1_000_000)), 1_000_000)` (1 μ m grid) and evaluate the orient3d determinant as an exact rational; the sign of the result is exact. **Do NOT use gmpy2.mpfr** — it is multi-precision float and remains subject to rounding for truly ambiguous predicates; using it for orient3d gives no more reliability than `float64` at any precision an implementer is likely to set.
- (b) **Shewchuk’s adaptive predicates** via a small `ctypes` binding (`predicates.so` — Shewchuk’s C source is < 1000 lines and compiles standalone with `gcc -O2`). This is the gold standard and is what the triangle library uses internally.

If neither is available, log a warning and treat the ambiguous case as non-crossing; CFM inputs are not adversarial and the failure mode is at most a missed near-tangency (no false positives). – Explicitly forbid the coplanar case ($|n_A \cdot n_B| > 1 - 1e-6$ AND $|d_A - d_B| < 1$ m): coplanar surfaces should not occur in CFM data (different geological structures); raise.

Acceptance criteria

- `tri_tri_intersect_3d` on two 90°-crossing 1m squares returns one segment per pair of intersecting triangles, segment endpoints $\leq 1e-9$ m from the geometric truth, in ≤ 2 of the 4 triangles.
- `chain_segments` on the 90°-cross test produces exactly one polyline of length ≥ 1.0 m (the full crossing edge).
- `scan_cross_fault_crossings("output/all8_2000m/stl", [all 8])` returns the README-documented crossing-pair counts to within $\pm 10\%$ (Mill $\times$ SBMT-SAF ≈ 163 , Mission $\times$ SAF ≈ 135 , Pinto $\times$ Mill ≈ 42 , etc.).
- Unit test for the false-positive case: two surfaces 100 m apart with similar normals $\rightarrow 0$ segments returned.
- Coverage on `eps_orient` ambiguity branch: synthetic case with a vertex within `eps_orient * edge2` of the other plane $\rightarrow$ either `gmpy2` resolves it or the warning fires.
- Run time of `scan_cross_fault_crossings` on the 8-fault 2000m set: ≤ 30 s on a laptop.

Edge cases to handle

- **Triangle B's edge lies in A's plane (positive-length 1-D intersection).** This is not a zero-area case — the edge is a real 1-D segment in 3-D and must be emitted. Detect it before running the main interpolation: if any pair of B's vertices satisfies `|d_B_i| < threshold` AND `|d_B_j| < threshold` (with the dimensionally-correct threshold per §Numerical robustness), clip the connecting edge against triangle A's interior using A's barycentric coordinates and emit the resulting sub-segment. Symmetric handling for A's edge in B's plane. Rare in raw CFM data (different geological structures aren't built in shared planes); possible in cascade phase if Phase 2's edge-pierce propagation produces a new edge that happens to lie in another fault's plane — log loudly, do not silently drop.
 - **Common vertex.** If A and B share a vertex (not in our use case but possible after Phase 2 cascading), the "intersection segment" has zero length; emit `nondegenerate=False`.
 - **Polyline branches at a T-junction** (rare in pairs, common with 3+ faults). Phase 1 only handles pairs; chaining produces 1 polyline per connected component, so a T-junction in the 3-fault case manifests as 3 polylines meeting at a point and is fine.
-

Phase 2 — Constrained re-triangulation

Goal

After this phase, each per-fault STL has been re-triangulated so that every cross-fault intersection polyline is a sequence of mesh edges on both faults. The pair of post-split STLs is a conforming PLC: HXT will accept it.

Files to create

- `mesh/conformalize_faults.py` (driver + library).
- `mesh/tests/test_conformalize_faults.py`.

Public API

```
def conformalize(
    in_stl_dir: Path,
    out_stl_dir: Path,
    included: List[str],
    *,
```

```

    snap_m: float = 1e-2,
    drop_short_segment_frac: float = 5e-2, # of triangle's mean edge length
) -> dict:
    """Returns the report dict (intersection_report.json contents).

    Note: per-parent corner-snap was removed in v2 (review P2-006);
    every polyline pierce is inserted as a Steiner vertex
    unconditionally so that fault A and fault B agree on the
    polyline geometry exactly. Slivers are guarded upstream by
    `eps_min_seg_len_m` (Phase 1) and `drop_short_segment_frac`."""

```

Algorithm

The job: every cross-fault polyline must appear as a sequence of mesh edges on **both** faults it lies on. We do this one pair of faults at a time, splitting one parent triangle at a time.

Outer loop

```

for each unordered pair of faults (A, B):
    polylines = chain_segments(tri_tri_intersect_3d(A, B))
    for each polyline P:
        split_triangles_on_one_fault(A, P)      # step (1)
        split_triangles_on_one_fault(B, P)      # step (1)
        propagate_edge_pierces_to_neighbours(A) # step (2)
        propagate_edge_pierces_to_neighbours(B) # step (2)
    write_per_fault_stl(A); write_per_fault_stl(B)
# cascade: 3+ faults: re-scan after each pair (step (3))

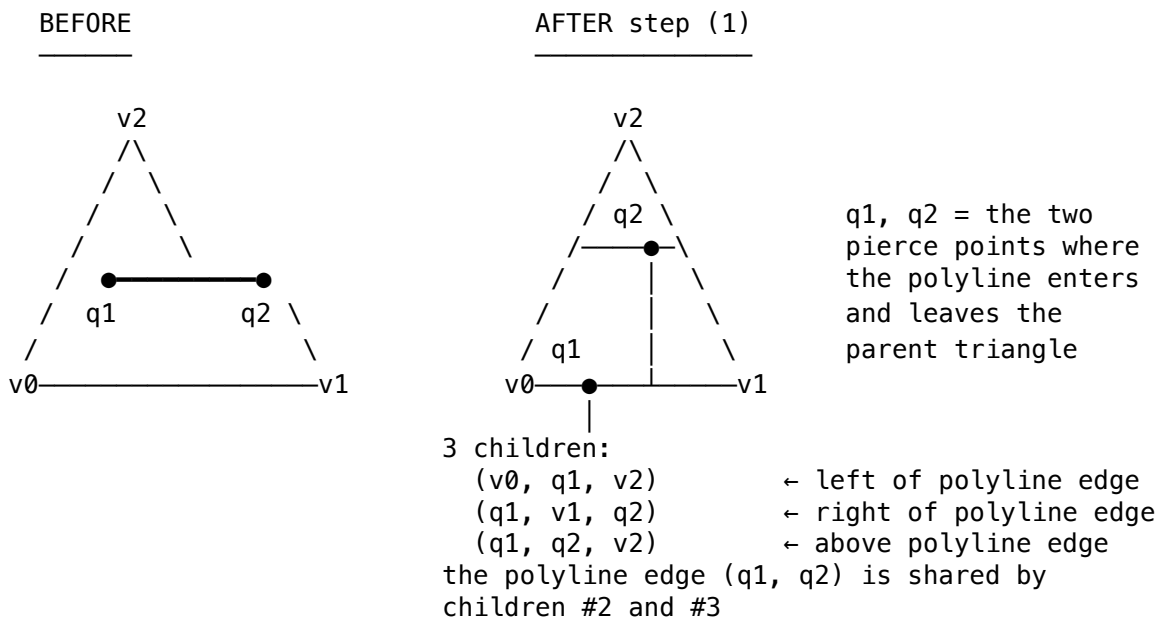
```

Step (1) — split one parent triangle For each parent triangle pierced by polyline P:

sub-step	what	why
a. project	parent's 3 vertices + P's pierce points → 2-D	constrained Delaunay is a 2-D operation
b. (no per-parent snap)	Insert every pierce as a Steiner vertex unconditionally	Per-parent corner snapping is dropped : the same polyline pierce can be snap-distance from a vertex of fault A's parent without being snap-distance from any vertex of fault B's parent (the two faults have different vertex layouts), which would silently break cross-fault polyline-edge coincidence. Slivers from short polyline segments are caught upstream by <code>eps_min_seg_len_m</code> (Phase 1 step 7) and <code>drop_short_segment_frac</code> ; what survives is guaranteed long enough to constrain.

sub-step	what	why
c. CDT	call <code>triangle.triangulate({verts, segments}, 'p')</code>	flag 'p' only — 'q' would insert Steiner points on the constraint and re-break conformality
d. assert	every input segment endpoint is in the output vertex list unchanged, every input segment is in the output segment list 1:1	guard against triangle silently subdividing
e. unproject	2-D children → 3-D via the inverse projection	back to mesh space
f. tag	every child inherits the parent's fault short-name	preserve provenance

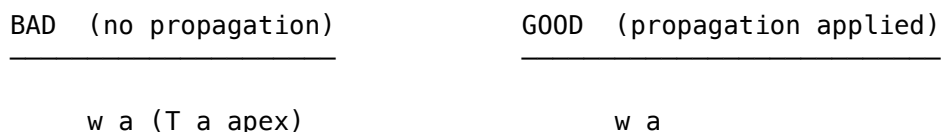
Picture (one parent triangle, one polyline crossing it — canonical case: polyline enters one edge, exits another, no interior pierce):

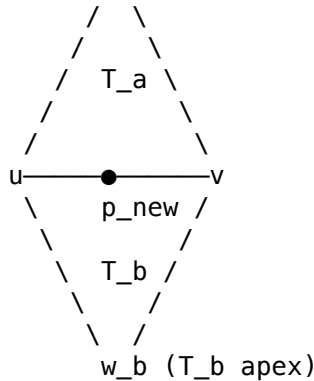


Number of children = 2 + (# pierce points strictly interior to the parent). Canonical case shown here: 0 interior pierces (both q1, q2 on edges) => exactly 3 children. A polyline that enters through an edge and terminates inside the parent (e.g., at a fault boundary) gives 1 interior pierce => 4 children. The geometry, not a fixed count, determines how many.

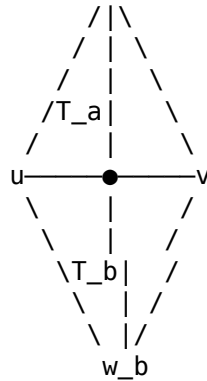
Step (2) — propagate edge-pierces to in-fault neighbours This is the step that prevents T-junctions. **Skipping it = HXT rejects the input** (same PLC error as the existing all-8 build).

When step (1) adds a vertex p_{new} on the boundary edge $e = (u, v)$ of parent T_a , the neighbour T_b sharing edge e (in the same fault) must also acquire p_{new} as a vertex — otherwise T_b 's edge passes through p_{new} without acknowledging it (a T-junction):





T_b is one undivided triangle even though p_{new} lies on its shared edge → T-junction.



T_b is split into two children: (u, p_{new}, w_b) and (p_{new}, v, w_b). Same for T_a: (u, p_{new}, w_a) and (p_{new}, v, w_a). No T-junction.

only ● = p_{new}

Algorithm for step (2):

```
for each edge e = (u, v) of the fault:
    pierces_on_e = sorted list of new vertices that landed on e (could be 0)
    for every triangle T whose edge is e (inside this fault):
        if T was re-triangulated by step (1) and already contains pierces_on_e:
            continue
        else:
            split T into n+1 children by walking along edge e
            and emitting (u, p1, w), (p1, p2, w), ..., (pn, v, w)
            where w is T's vertex opposite e, and p1..pn are pierces_on_e
```

Step (3) — cascade across 3+ faults When fault A has pierces from both polyline P_{AB} and polyline P_{AC}, process them sequentially:

```
process polyline PAB: → A's mesh becomes Aafter_AB
process polyline PAC: ← MUST re-run tri_tri_intersect_3d(Aafter_AB, C)
                        to get fresh pierce-triangle indices
                        (the original indices into A reference triangles
                        that no longer exist)
```

Rule: **between cascade passes, re-scan**. Don't try to remap indices via parent-child bookkeeping; the rescan is 0(pair-scan-cost) per pass and is the simpler, safer choice.

Within a single cascade pass, all constraint segments touching one parent triangle (typically multiple consecutive segments of the same polyline that re-enter the parent through different edges) are collected and passed in **one** triangle.triangulate({segments: ...}, 'p') call. Mixing the two rules is correct, and the distinction matters:

- **Within-pass, multiple segments → one CDT call.** The triangle library handles multiple constraint segments natively; this is the cheapest and most robust path.
- **Across passes (different fault pairs P_{AB} and P_{AC} touching the same parent of fault A)** → re-scan + sequential CDT calls. After P_{AB} is processed, the original parent no longer exists; P_{AC}'s pierces must reference the post-P_{AB} children. The re-scan finds those new pierce-triangle indices.

Outputs

- <out_stl_dir>/<short>.stl — per-fault re-triangulated STL.

- <out_stl_dir>/triangle_to_fault.json:

```
{
  "schema_version": 1,
  "faults": {
    "safs_sbmt_millcreek": {"n_triangles": 901, "range": [0, 901]},
    "safs_sbmt_saf": {"n_triangles": 1490, "range": [901, 2391]}
  },
  "intersection_polylines": [...]
}
```

- <out_stl_dir>/intersection_report.json — per polyline: participating faults, # vertices, # children before/after on each side; aggregate stats.

Robustness rules

- Drop any output sub-triangle whose area $< 1e-6 \text{ m}^2$ (zero-area protection — same as `_drop_degenerate` in `ts_to_stl.py`).
- Drop any polyline segment shorter than `drop_short_segment_frac * mean_edge_in_parent`; record in the report.

Validation gates (per fault and per pair)

The previous “find_overlap_pairs returns []” and “tri_tri_intersect_3d returns zero-length segments” gates were tautologies: `find_overlap_pairs` requires coplanar same-side overlap so it trivially returns [] regardless of conformalization (children of different parents are not coplanar), and post-conformal child pairs share a full edge along the polyline (not zero-length crossings). Both gates pass on broken implementations. Replace with three substantive gates:

1. **Manifold gate (per fault).** For each post-split fault: every edge is shared by ≤ 2 triangles within the fault, and no vertex lies in the open interior of any other edge to within $1e-3 \text{ m}$ (the T-junction check from step 4 above). Implemented as `is_manifold_no_t_junctions(V, T) -> (bool, list[edge])` in `fault_intersect.py`.
2. **Polyline edge-coincidence gate (per pair).** For every intersection polyline P from fault A $\times$ B: every consecutive vertex pair ($P[i], P[i+1]$) must appear as an edge in fault A’s post-split mesh AND in fault B’s post-split mesh. A mismatch indicates a conformalizer bug (a pierce point made it onto only one side). Implemented as `verify_polyline_in_mesh(V, T, polyline, match_tol_m=2e-2)` in `conformalize_faults.py`. A mesh vertex is considered to “match” a polyline point when their Euclidean distance is below `match_tol_m`. The default `match_tol_m = 2 * snap_m = 2 \text{ cm}` is large enough to absorb 2-D-projection / 3-D-unprojection round-off (sub-mm at SAFS coordinate scales) and far smaller than the minimum CFM inter-vertex spacing ($\sim 100 \text{ m}$ at 2000m resolution), so it cannot cause false positives on real data.
3. **Interior-crossing gate (per pair).** Run a new `tri_tri_intersect_3d_interior_only(V_A, T_A, V_B, T_B)` which filters out triangle pairs (i, j) that share one or more vertices (these are the polyline-edge coincidences and are correct, not failures). The post-split set must report 0 interior crossings. The original `tri_tri_intersect_3d` is the wrong primitive for this check because it reports shared-edge segments as full-edge-length crossings.

Acceptance criteria

- **Synthetic test:** two 90°-crossing 1m squares (2 tris each $\rightarrow$ expected 4 tris each post-split). After conformalize: 4 tris each, intersection edge appears as exactly one new edge in each fault; the **interior-crossing gate** reports 0 interior crossings (shared-edge pairs along the polyline are explicitly excluded).

- ❑ **Manifold gate** (P-004): each post-split fault has 0 T-junctions; no vertex lies in the open interior of any other edge within $1e-3$ m.
- ❑ **Polyline edge-coincidence gate** (P-005): every consecutive polyline vertex pair appears as an edge in both faults' post-split meshes.
- ❑ **Real fixture**: Mill Creek $\times$ SBMT-SAF at 2000m. Pre-split tris: $794 + 1346 = 2140$. Post-split tris $\approx 2140 + 2 \times N_{\text{pierce_total}}$ (each polyline vertex inserted as a Steiner point on each side generates ~ 2 extra triangles). The **interior-crossing gate** reports 0 interior crossings on the post-split STLs (shared polyline edges are excluded).
- ❑ All post-split per-fault STLs survive `audit_ts_quality.py --include-fault <short>` with `n_overlap_pairs == 0` and `n_nonmanifold_edges == 0`. (These checks remain useful as sanity gates but are NOT a substitute for the manifold gate above — `n_nonmanifold_edges` only catches edges shared by 3+ triangles, not T-junctions.)
- ❑ `triangle_to_fault.json` written, schema-valid, and per-fault counts sum to total combined.
- ❑ Run time on Mill Creek $\times$ SBMT-SAF: ≤ 60 s on a laptop.

Edge cases to handle

- **Pierce point at a triangle vertex**: insert as a Steiner vertex anyway (per the no-snap rule, P2-006). The constrained Delaunay step accepts coincident-with-existing-vertex inputs and emits zero new vertices for them — so the pierce is correctly registered without per-parent snap heuristics. If the pierce point coincides with a vertex of both fault A's parent AND fault B's parent (extremely rare for raw CFM data), the same Steiner-insert is a no-op on both sides and conformality is preserved.
- **Polyline endpoints that fall on the free surface** ($z = -\text{clearance}$): Phase 1 step 9 has already clamped any too-shallow segment endpoints exactly to $z = -\text{clearance}$; the conformalizer treats the resulting endpoint as a normal pierce point (Steiner vertex on the parent triangle's edge or interior, depending on geometry). The free-surface trace is then a sequence of fault triangles whose top edge lies on $z = -\text{clearance}$; `validate_msh.py` check 5 already permits these to have `n_trace_with_1_tet`.
- **Polyline endpoints that fall on the fault boundary** (an open-surface edge of either fault): same as above — terminate the polyline at the fault boundary.
- **A parent triangle hit by multiple constraint segments within one cascade pass** (e.g., a single polyline that enters, exits, and re-enters the same parent across different edges): re-triangulate with all relevant segments as constraints in **one** `triangle.triangulate({segments: ...}, 'p')` call. The triangle library handles multiple constraint segments natively. This is not the same as the across-pass cascade case — when polylines `P_AB` and `P_AC` both touch the same parent of fault A, they belong to different cascade passes (Step (3)) and are processed sequentially with a re-scan in between, not in one CDT call. The difference: within-pass, the constraint segments are all known up front because they come from one `tri_tri_intersect_3d` scan; across-pass, `P_AC`'s pierces aren't even computed until after `P_AB` has changed the triangulation.

Phase 3 — Tube-uniform size field

Goal

After this phase, the bulk mesh has a uniform `res_f` shell of configurable thickness around every fault, and coarsens to `res_ff` beyond that shell over `ramp_dist`.

Files to modify

- `mesh/safs.geo` — Threshold field `DistMin = tube_radius` instead of 0.

- mesh/generate_safs_mesh.py — --tube-radius CLI flag (default 5000 m); validate tube_radius >= 0 and tube_radius < buf_x / < buf_y (otherwise the tube engulfs the box buffer and there is no far-field).
- mesh/validate_msh.py — new check_11_tube_uniformity.

See mesh/REVIEW.md R-004 for the concrete diff.

Acceptance criteria

- ☐ Smoke test (single fault, 1000m, tube=4000m): mean fault edge $\approx$ 1000 m (already passing); mean within-tube tet edge $\approx$ 1000 m $\pm$ 100 m (new check); far-field mean $\approx$ res_ff $\pm$ 10%.
- ☐ 2-fault target: same metrics with both faults sharing the tube.
- ☐ When tube_radius = 0, behavior matches today's code (back-compat — single-fault smoke still passes 10/10).

Edge cases to handle

- **Tube engulfs the entire bulk box** (user passed tube_radius > domain dimension / 2): driver hard-fails with a clear message.
- **Two faults closer than 2 $\times$ tube_radius**: tubes overlap — fine; size field takes the minimum (Field[2].SizeMin = res_f everywhere inside either tube).

Phase 4 — End-to-end 2-fault target

Goal

After this phase, bash run_two_crossing_2000m.sh runs the full pipeline (audit $\rightarrow$ ts_to_stl $\rightarrow$ conformalize $\rightarrow$ mesh $\rightarrow$ provenance $\rightarrow$ validate) and produces a 10/10-validated mesh.

Files to create

- mesh/run_two_crossing_2000m.sh (see mesh/REVIEW.md R-005 for the full script).

Files to modify

- mesh/generate_safs_mesh.py:
 - Detect triangle_to_fault.json in --stl-dir; if present, treat as conformal-input and skip the cross-fault scan; else run scan_cross_fault_crossings(...) and refuse non-conforming multi-fault input.
 - Wire --tube-radius (Phase 3).
 - When operating on a --stl-dir containing per-fault conformal STLs (not a single combined STL), _combine_stls snap-dedups those at the configurable tolerance; the combine step is now cheap because the inputs are already conforming.
- mesh/safs.geo:
 - Mesh.Algorithm3D = 10 (HXT) for conformal input. Keep Algorithm3D = 4 only as a manually-selected fallback, gated on a CLI flag rather than the default.
- mesh/write_fault_provenance.py:
 - When triangle_to_fault.json is supplied, use it directly (range-based attribution); fall back to the existing KDTree path only when the conformalizer was not run.
- mesh/validate_msh.py:
 - check_5_internal_interface is unchanged (already correct); expected to PASS now that input is conforming.
 - Add check_11_tube_uniformity (Phase 3).

Acceptance criteria

- ☐ `bash run_two_crossing_2000m.sh` exits 0 with 10/10 checks passed (or 11/11 with the new tube check).
 - ☐ In particular, check `_5_internal_interface`: `n_internal_with_2_tets ≈ #fault_tris - #trace_tris`, `n_offending = 0`. Compare to today's all-8 number (`n_offending = 4917`, every triangle wrong).
 - ☐ Provenance: every post-split triangle is attributed to the correct CFM fault (Mill Creek vs SBMT-SAF) — no KDTree mis-attribution at the intersection curve.
 - ☐ Visual check in ParaView: continuous fault surface across the crossing, no T-junctions, with a visible uniform-size shell around it.
 - ☐ End-to-end runtime on a laptop: ≤ 90 s.
-

Phase 5 — Cascade to 3+ faults (out of scope; verification only)

Goal

Confirm Phase 2's cascading logic actually scales. Not a code change — a verification pass.

Files to modify (verification only)

- Add a new “crossing subset” recipe to `run_subsets_2000m.sh` combining 3 faults including at least 2 crossing pairs (e.g., Mill Creek + SBMT-SAF + Mission Creek SBMT — $163 + 41 = 204$ crossing pairs, all between SEAS-grade individual faults).

Acceptance criteria

- ☐ 3-fault crossing subset passes 11/11 validator checks.
- ☐ **Cascade actually cascades**: `intersection_report.json` records, for each pierce, whether its parent triangle existed in the initial per-fault triangulation or was created by an earlier cascade pass. The 3-fault target must produce at least one pierce on a child triangle that did NOT exist pre-cascade — proves the re-scan rule (Step (3)) actually operated on the post-split mesh, not the original.

The all-8 build remains gated on Pinto + Banning CFM-source defects (README §Caveats 2). Resolving those is independent of this plan.

Testing strategy

Phase	Synthetic test	Real fixture	Validator gate
1	90°-cross squares; 100 m parallel; near-tangent	Mill × SBMT-SAF crossing-pair count vs README	scan returns expected pairs

Phase	Synthetic test	Real fixture	Validator gate
2	90°-cross squares + cascade with a 3rd plane	Mill × SBMT-SAF post-split STLs pass all three Phase-2 gates	(a) manifold gate per fault, (b) polyline edge-coincidence gate per pair, (c) interior-crossing-only gate per pair (see §Validation gates for the definitions; the older <code>find_overlap_pairs</code> / “scan returns 0” gates are tautologies and are NOT used here)
3	None (size-field config only)	Single-fault smoke retains 10/10; 2-fault tube uniformity	<code>check_11</code> passes
4	None (integration)	<code>run_two_crossing_20000/11</code> (or 11/11) end-to-end	including <code>check_5</code>
5	None	3-fault crossing subset	11/11

The synthetic tests live under `mesh/tests/` and run via `pytest miniapps/seas/safs/mesh/tests/`. The real fixtures run via the wrapper shell scripts and write `validation_report.txt` to the run output directory.

Risk assessment

1. **Constrained Delaunay robustness** at near-tangent intersections. The triangle library uses Shewchuk’s exact predicates and is well-tested, but our 2-D projection introduces float64 round-off on the order of $1e-9 \times \text{max_coord}$. Mitigation: project into the parent triangle’s local frame (orthonormal basis from triangle normal + one edge direction) so coordinates are $O(\text{edge_length})$, not $O(\text{domain_size})$. This gives $\approx 1e-6$ m precision regardless of the global frame’s magnitude.
2. **Cascading creates many tiny sub-triangles** at the crossing knot. If 8 faults all meet within a 100 m radius (San Gorgonio), the knot can produce $O(100)$ tiny triangles per fault. Mitigation: `drop_short_segment_frac = 0.05` already removes the worst; if slivers persist into the bulk mesh, raise `Mesh.OptimizeNetgen = 1` and `Mesh.OptimizeThreshold = 0.3` (already on in `safs.geo`). Run quality check 10 to verify.
3. **triangle Python wrapper not in pythonenv**. Verify before coding; if absent, install via `pip install triangle` in the env. (Pure-Python fallback using `scipy.spatial.Delaunay` + manual edge constraint enforcement is possible but more code; defer.)
4. **CFM input has duplicated vertices across faults**. `ts_to_stl.py` already snaps within-fault duplicates; cross-fault duplicates are harmless to Phase 1 (the intersection algorithm is purely geometric) and harmless to Phase 2 (each fault is processed independently).
5. **The free-surface trace**. Polylines may terminate at the free-surface clearance plane ($z = -100$ m in default config). `validate_msh.py:check_5` already permits boundary triangles to have 1 adjacent tet; the conformalizer must not insert Steiner vertices above $z=0$ (would re-introduce R-002’s category of bug). Add a guard: **CLAMP** (not drop) any pierce point

with $z > -\text{clearance}$ to exactly $z = -\text{clearance}$ via linear interpolation along the segment. Drop only the segment of an intersection polyline that lies wholly above $-\text{clearance} + \text{slop}$ (the segment is geometrically above the bulk volume's free surface and has no role). Dropping individual pierce points instead of clamping would silently truncate polylines and leave dangling intersection edges below the surface — implemented in Phase 1 step 9.

6. **Performance for the all-8 set** (Phase 5). Naive $O(N^2)$ pair scan on 8 faults with triangle counts {1346, 794, 504, 449, 346, 234, 610, 639} produces $\approx 4.1\text{M}$ cross-fault pairs (computed as $(\sum n)^2 - \sum n^2 / 2 = (24.2\text{M} - 3.85\text{M}) / 2$; only the upper triangle of the bipartite-pair matrix is scanned). Numpy-vectorized naive scan handles this in single-digit seconds; AABB tree is unlikely to be needed at the 2000m resolution. Defer. Out of scope for the 2-fault milestone.

Out of scope (explicitly)

- OCC / BooleanFragments / manifold / TetGen alternatives. Documented dead-ends in README §Caveats and §Roadmap.
- Pinto Mountain (safs_pmfz_pinto) and Banning standalone (safs_mult_banning) — CFM-source-defective; require source triangulation repair, not intersection handling.
- Per-fault different `res_f` / `tube_radius`. Single global value is fine for the 2-fault target.
- Coupled solver work (renaming faults to per-fault tags in `safs.geo`, wiring multiple `Physical Surface` IDs through `fault/`). PLAN.md Phase 6 already deferred this; it remains deferred.
- TPV-style validation against analytical solutions on this geometry. CFM is data, not a benchmark.